KOENIG

step forward



Kotlin Collections

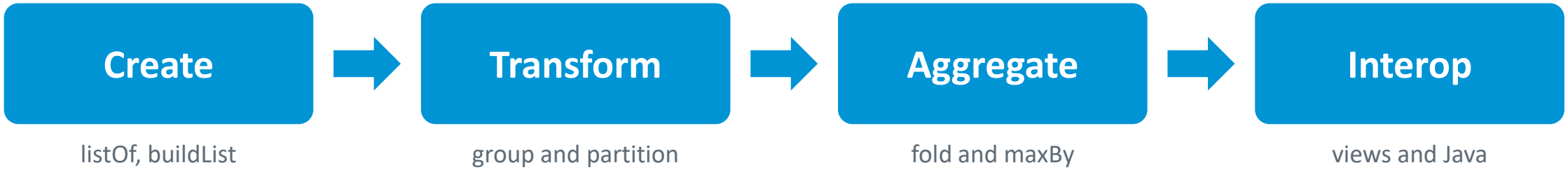
Kotlin for Java Developers

What You Will Learn

1. Creating collections: listOf, mapOf, buildList
2. Transformations: map, filter, flatMap, groupBy
3. Aggregations: fold, sumOf, maxByOrNull
4. Sequences and lazy evaluation
5. Read-only versus mutable interfaces
6. Kotlin collections versus Java Streams

Demo Scenario: The ShopKart Report in Kotlin

Module 3 built ShopKart's sales report with Java Streams. We rebuild it with Kotlin collection functions on the same sample day, then look at lazy sequences, read-only views and the Java boundary.



We will cut

- **stream().collect**
- **Hand-made maps**
- **Hidden mutation**

Why Kotlin Collections?

```java
// Java: wrap, stream, collect
List<String> upper = names.stream()
    .filter(n -> n.length() > 4)
    .map(String::toUpperCase)
    .collect(Collectors.toList());

Map<String, List<Sale>> byCat =
    sales.stream().collect(
        Collectors.groupingBy(Sale::category));
```

```kotlin
// Kotlin: call directly on the collection
val upper = names
    .filter { it.length > 4 }
    .map { it.uppercase() }

val byCat = sales.groupBy { it.category }
```

- **No stream():** operations live on List, Set and Map.
- **No collect():** every step returns a normal collection.
- **Same JVM classes:** Kotlin adds a rich API on java.util types.

Same results as a Stream pipeline, with far less ceremony

Creating: listOf, setOf, mapOf

```kotlin
val names = listOf("Asha", "Ravi", "Meena")
val tags  = setOf("kotlin", "java", "kotlin")
val stock = mapOf("pen" to 10, "book" to 3)

println(names[0])           // Asha
println(tags)               // [kotlin, java]
println(stock["pen"])       // 10
println(stock["zzz"])       // null
println(stock.getOrDefault("zzz", 0))   // 0
```

```java
// Java equivalents
List<String> names =
    List.of("Asha", "Ravi", "Meena");
Set<String> tags = Set.of("kotlin", "java");
Map<String,Integer> stock =
    Map.of("pen", 10, "book", 3);
```

- **Duplicates in setOf** are dropped; order is kept (LinkedHashSet).
- **to** builds a Pair: the entries of mapOf.

Missing map key gives null, not an exception

Creating: Mutable and buildList

```
val log = mutableListOf("start")
log.add("loaded")
log += "done"                       // plusAssign

val ids = mutableSetOf(1, 2)
val counts = mutableMapOf("a" to 1)
counts["b"] = 2                     // put

val squares = buildList {
    for (i in 1..5) add(i * i)
}                                   // read-only result
println(squares)                    // [1, 4, 9, 16, 25]
```

mutableXOf

Gives a MutableList, MutableSet or MutableMap you can change.

buildList

Mutate inside the lambda, get a read-only list back. Also buildSet and buildMap.

Prefer val + read-only types; use mutable ones locally

Operators: plus, minus, +=

```
val a = listOf(1, 2)
val b = a + 3                    // new list [1, 2, 3]
val c = b - 1                    // new list [2, 3]

var d = listOf(1)                // var + read-only
d += 2                           // d = d + 2, new list

val e = mutableListOf(1)         // val + mutable
e += 2                           // e.add(2), same list
```

var + List

+= creates a NEW list and reassigns the variable.

val + MutableList

+= adds in place. Pick one style, not both.

val x = listOf(1); x += 2 is a compile error: be explicit

Demo: Creating Collections

What we build

- **listOf, setOf, mapOf:** read-only and short.
- **mutableListOf, buildList:** mutable inside.
- **plus, minus, +=:** and the val error.

How the concept helps

- **No stream():** call on the collection.
- **Missing key:** null, not an exception.
- **One style:** val + mutable, or var + read-only.

Transform: map and filter

```
val nums = listOf(1, 2, 3, 4, 5, 6)

nums.map { it * 10 }
// [10, 20, 30, 40, 50, 60]

nums.filter { it % 2 == 0 }
// [2, 4, 6]

nums.filter { it % 2 == 0 }.map { it * 10 }
// [20, 40, 60]
```

```
// Java Stream
nums.stream()
    .filter(n -> n % 2 == 0)
    .map(n -> n * 10)
    .collect(Collectors.toList());
```

- **it:** the implicit name of a single lambda parameter.

Each call returns a new List; the original is untouched

Transform: flatMap

```
val orders = listOf(
    listOf("pen", "book"),
    listOf("desk"),
    listOf("pen", "lamp"))

orders.map { it.size }
// [2, 1, 2]

orders.flatMap { it }
// [pen, book, desk, pen, lamp]

orders.flatten().distinct()
// [pen, book, desk, lamp]
```

map

one result per element: list of lists stays nested.

flatMap

each element becomes a list, all are joined into one.

Java: stream().flatMap(l -> l.stream()) becomes flatMap { it }

Transform: associate and associateBy

```
data class Sale(val id: Int, val orderId: String, val customer: String,
    val region: String, val product: String, val category: String, val amount: Double)

val byId = sales.associateBy { it.id }
// Map<Int, Sale>

val names = sales.associate { it.id to it.product }
// Map<Int, String>

val lens = listOf("pen", "book").associateWith { it.length }
// {pen=3, book=4}
```

- **associateBy:** key from the element, value is the element.
- **associate:** you build each Pair yourself.
- **Duplicate keys:** the last element wins, nothing throws.

Transform: groupBy

```
val byCategory: Map<String, List<Sale>> =
    sales.groupBy { it.category }
// ELECTRONICS: ids 1, 2, 8, 10   FURNITURE: 3, 4, 7, 11

val totals = byCategory.mapValues { (_, items) ->
    items.sumOf { it.amount }
}
// {ELECTRONICS=132397.0, FURNITURE=29600.0, STATIONERY=1300.0}

val counts = sales.groupingBy { it.category }.eachCount()
// {ELECTRONICS=4, FURNITURE=4, STATIONERY=3}
```

```
// Java
Collectors.groupingBy(
  Sale::category,
  Collectors.summingDouble(
      Sale::amount))
```

- **groupBy:** Map of key to List.
- **mapValues:** transform each group.

groupBy + mapValues replaces groupingBy with a downstream collector

Transform: partition

```kotlin
val (big, small) = sales.partition { it.amount >= 5000 }

println(big.map { it.product })
// [Laptop, Desk, Chair, Laptop, Desk]
println(small.size)
// 6

// partition returns a Pair<List, List>
val pair: Pair<List<Sale>, List<Sale>> =
    sales.partition { it.category == "STATIONERY" }
```

```java
// Java
Map<Boolean, List<Sale>> m =
 sales.stream().collect(
  Collectors.partitioningBy(
   s -> s.amount() >= 5000));
m.get(true);
```

One pass, two lists, destructured straight into names

Demo: Transforming the Sample Day

What we build

- **Sale rows:** flatMap of the Java orders.
- **associateBy, groupBy:** lookups and totals.
- **partition:** big and small sales.

How the concept helps

- **Same numbers:** as the Java report.
- **Two steps:** group, then reduce.
- **One pass:** two lists.

Aggregations: sum, count, fold

```
val amounts = sales.map { it.amount }

amounts.sum()                           // 163297.0
sales.sumOf { it.amount }               // 163297.0
sales.count { it.category == "STATIONERY" }   // 3
sales.size                              // 11

sales.fold(0.0) { acc, s -> acc + s.amount }
// 163297.0

amounts.reduce { a, b -> maxOf(a, b) } // 65000.0
```

sumOf

one lambda to the number you want to add up.

fold / reduce

combine all elements into one value, step by step.

sumOf replaces mapToDouble(...).sum()

fold versus reduce

```
val xs = listOf(3, 1, 4)

xs.fold(10) { acc, x -> acc + x }   // 18
xs.reduce { acc, x -> acc + x }     // 8

emptyList<Int>().fold(10) { a, x -> a + x }  // 10
// emptyList<Int>().reduce { a, x -> a + x }
//   -> UnsupportedOperationException
emptyList<Int>().reduceOrNull { a, x -> a + x }  // null
```

fold

takes an initial value. Result type can differ. Safe on empty.

reduce

starts from the first element. Fails on an empty collection.

maxByOrNull and Friends

```
val biggest = sales.maxByOrNull { it.amount }
println(biggest?.product)          // Laptop

val none = emptyList<Sale>().maxByOrNull { it.amount }
println(none)                      // null

val label = biggest?.customer ?: "no data"

sales.minByOrNull { it.amount }?.product  // Notebook
listOf(2, 9, 4).maxOrNull()        // 9
sales.sortedByDescending { it.amount }.take(3)
```

```
// Java returns an Optional
sales.stream()
  .max(Comparator
    .comparingDouble(
      Sale::amount))
```

- **OrNull:** null for empty, no Optional.

The OrNull suffix means: empty input gives null, not a crash

Demo: Aggregations

What we build

- **sumOf, count, fold:** totals of the sales.
- **fold, reduce:** empty list behaviour.
- **maxByOrNull:** null when empty.

How the concept helps

- **No Optional:** a nullable result.
- **fold:** safe on empty input.
- **Elvis:** a default for no data.

Eager Collections vs Sequences

List: eager, step by step

- filter ALL elements
- then map ALL survivors
- then take first

Sequence: lazy, per element

- element 1: filter, map
- element 2: filter, map
- stop when the result is found

Same operators, different order of work

Sequence: Evaluation Order

```
(1..6).toList()                  // eager
    .filter { println("filter $it"); it % 2 == 0 }
    .map    { println("map    $it"); it * 10 }
    .first()

(1..6).asSequence()              // lazy
    .filter { println("filter $it"); it % 2 == 0 }
    .map    { println("map    $it"); it * 10 }
    .first()
```

```
Eager output:
filter 1  filter 2  filter 3
filter 4  filter 5  filter 6
map    2  map    4  map    6
result = 20

Sequence output:
filter 1
filter 2
map    2
result = 20
```

- **Eager:** 9 lambda calls.
- **Lazy:** 3 lambda calls, stops at first.

This is the real output of the demo

Creating and Using Sequences

```
val s1 = listOf(1, 2, 3).asSequence()
val s2 = sequenceOf("a", "b")

val squares = generateSequence(1) { it + 1 }  // infinite
    .map { it * it }
    .take(5)                                  // intermediate
    .toList()                                 // terminal
// [1, 4, 9, 16, 25]

val pending = s1.map { println("never runs"); it }
// nothing printed: no terminal operation yet
```

Intermediate

map, filter, take: return another Sequence, no work done.

Terminal

toList, first, sum, count: trigger the whole pipeline.

No terminal operation, no work

When to Use a Sequence

Use Sequence

Large data, many chained steps, or early exit (first, take, any).

Use List

Small data, one or two steps, or you need the result more than once.

Cost of List

Every step builds a full intermediate list in memory.

Cost of Sequence

Per-element overhead; a sequence can be consumed again, redoing the work.

Measure before you switch; lists are fine for most code

Demo: Sequences and Evaluation Order

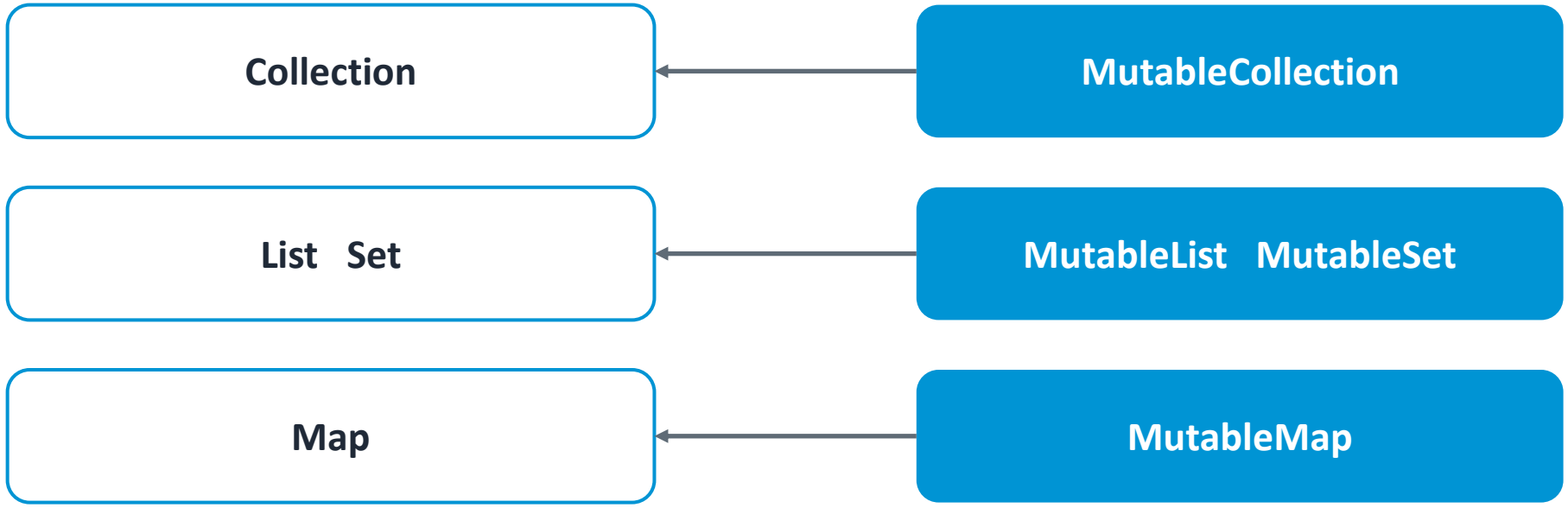
What we build

- **filter, map, first:** eager against lazy.
- **generateSequence:** infinite and lazy.
- **Reuse:** the work runs again.

How the concept helps

- **9 against 3:** lambda calls.
- **Terminal:** no terminal, no work.
- **Choice:** measure first.

Read-Only vs Mutable Interfaces



- **Read-only:** get, size, contains, iterate. No add or remove.
- **Mutable** extends read-only and adds add, remove, clear, set.
- **Arrows:** MutableList is-a List.

Mutability is part of the TYPE, checked by the compiler

Read-Only Is a View, Not Frozen

```
val backing = mutableListOf(1, 2, 3)
val view: List<Int> = backing

backing.add(4)
println(view)            // [1, 2, 3, 4]

// view.add(5)           // does not compile
(view as MutableList<Int>).add(5)   // compiles, works!
println(backing)         // [1, 2, 3, 4, 5]
```

A view

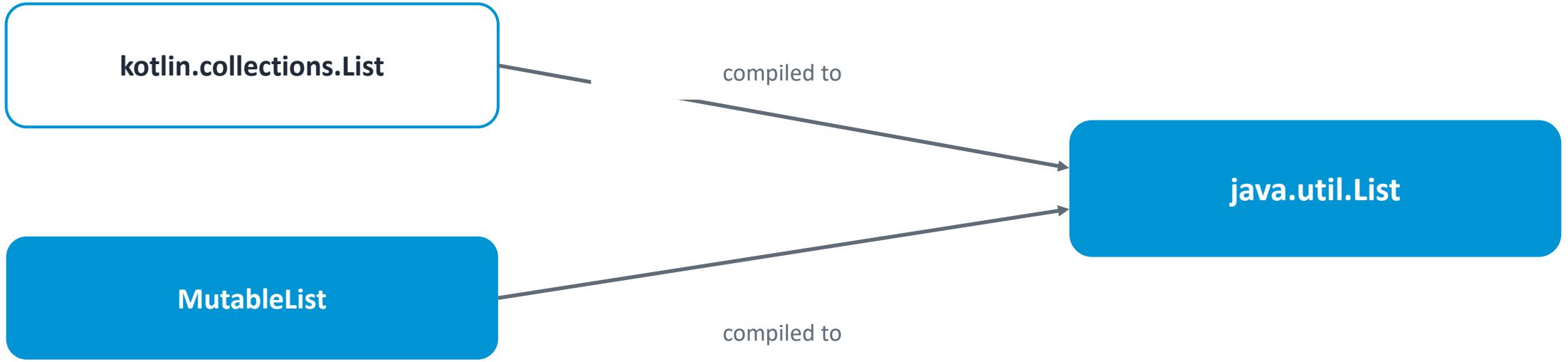
List<Int> only limits what YOU can call. The object underneath may still change.

A copy

toList() creates a new list that no longer follows the original.

Return toList() when a caller must never see later changes

Kotlin Types on the JVM



- **One runtime type:** both map to java.util.List; ArrayList, HashSet, HashMap are used underneath.
- **No conversion:** passing a Kotlin list to Java copies nothing.
- **So:** Java code can mutate a Kotlin read-only list at runtime.

The read-only rule is enforced by the Kotlin compiler only

Demo: Read-Only Views and Copies

What we build

- **List and MutableList:** one object.
- **Cart:** private list, safe getter.
- **javaClass:** java.util underneath.

How the concept helps

- **Compiler only:** the cast bypasses it.
- **toList():** a copy that stays.
- **Interop:** free, but unguarded.

Kotlin Collections vs Java Streams

	Kotlin collections	Java Streams
Start	none: call on the collection	stream()
Result	collection after every step	collect(...) at the end
Lazy?	only with asSequence()	always lazy
Empty max	null	Optional
Reuse	fully reusable	single use

Interop: Streams and Collections

```
import kotlin.streams.asSequence
import kotlin.streams.asStream

// Java stream -> Kotlin
val list = TagService.tagStream().toList()
val seq  = TagService.tagStream().asSequence()

// Kotlin -> Java stream
val st1 = names.stream()              // List is a java.util.List
val st2 = names.asSequence().asStream()
```

Stream to Kotlin

Stream.toList() or asSequence() (kotlin.streams).

Kotlin to Stream

list.stream() works directly; sequences need asStream().

Kotlin lists ARE java.util.Collection: stream() just works

Demo: Kotlin Collections and Java Streams

What we build

- **Java report:** ShopStreams.revenueByRegion.
- **Kotlin report:** groupBy, mapValues.
- **Stream reuse:** second count fails.

How the concept helps

- **Same output:** less ceremony.
- **asSequence, asStream:** the bridge.
- **Default:** collections; Streams for Java APIs.

Key Takeaways

1. **Types**
   List vs MutableList is compile-time only: a view, not frozen.

2. **Create**
   listOf, setOf, mapOf, mutableListOf, buildList.

3. **Transform**
   map, filter, flatMap, associateBy, groupBy, partition.

4. **Aggregate**
   sumOf, fold, reduce, count, maxByOrNull.

5. **Lazy**
   asSequence() changes evaluation order and saves work.



Happy Learning :)